RecLoop: Recurrent State Modeling for Looped Transformers

Cheng Liang[1], Tengfei Zhang[2], Jianyu Wu[1], Wenjie DU[3]
Changlian Ma[4], Ya Zhang[1], Yanfeng Wang[1,*]

[1] Shanghai Jiaotong University
[2] University of Science and Technology of China
[3] Nanyang Technological University [4] Nanjing University
[*] Corresponding author

Abstract

Looped language models offer a parameter-efficient way to scale computation by repeatedly applying shared Transformer blocks. Yet existing architectures commonly feed the terminal hidden state of one loop directly into the next. This naive handoff forces the same residual stream to act as both the current working representation and the sole carrier of earlier computation, encouraging later loops to revisit rather than selectively build on previous results. We introduce *RecLoop*, which separates these roles through explicit recurrent state modeling across loops. A shared residualized GRU accumulates a token-aligned memory at loop boundaries, and Dual-QKV Fusion combines this memory with the current working state in attention queries, keys, and values. Thus, each loop can retrieve accumulated history while all Transformer and memory-transition parameters remain shared across iterations. Controlled pretraining experiments across parameter scales and recurrent depths show that RecLoop consistently improves language modeling over matched vanilla looped Transformers, with scaling-law fits indicating approximately 11% fewer parameters at matched validation loss. Causal history knockouts further confirm that later loops use the accumulated recurrent state.

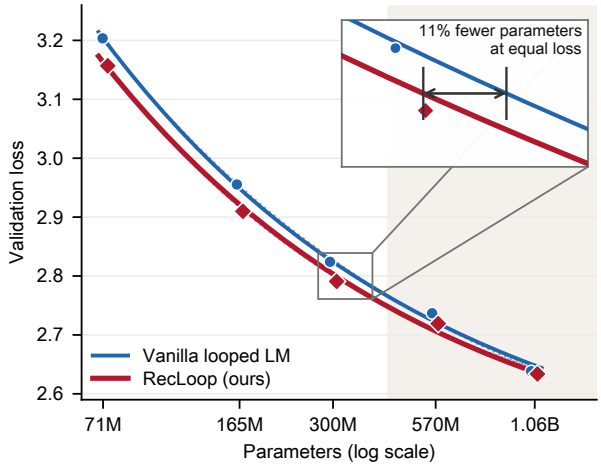


Figure 1: Language modeling loss on FineWeb-Edu (Penedo et al. 2024) as parameter count scales from 71M to 1.06B. All models are trained on 10B tokens. RecLoop achieves lower validation loss at every scale. The fitted scaling laws indicate that RecLoop requires approximately 11% **fewer parameters** than the vanilla looped LM to attain the same validation loss.

1 Introduction

Scaling model parameters, data, and compute has driven the progress of large language models, but continued parameter growth also raises training and deployment costs (Kaplan et al. 2020; Hoffmann et al. 2022). Looped language models offer a complementary scaling path: repeatedly applying shared Transformer layers increases computation without increasing parameter count and iteratively refines representations in continuous latent space (Dehghani et al. 2019; Lan et al. 2020). Recent work further shows that looped models can exploit additional recurrent steps for useful internal computation: they can learn iterative algorithms, perform multi-step reasoning in latent space, and improve their performance when allocated more test-time compute (Giannou et al. 2023; Yang et al. 2024; Saunshi et al. 2025; Geiping et al. 2025; Zhu et al. 2025). These capabilities position looped language models as a promising architecture for flexibly scaling computation under constrained parameter budgets.

Despite this potential, allocating more recurrent steps does not automatically yield proportional performance gains. Empirical studies report rapidly diminishing returns as loop count increases, with performance eventually saturating or even degrading in some settings (Geiping et al. 2025; Prairie et al. 2026; Zhu et al. 2025). Deeper recurrence can also amplify residual-state growth and make optimization more difficult (Wang et al. 2026; Prairie et al. 2026; Li et al. 2026a). These findings show that the value of recurrent depth depends on how computation across iterations is structured, rather than on loop count alone.

One potential bottleneck in current looped language models could lie in how historical information is organized across iterations. In the standard single-stream architecture, the terminal hidden state of one iteration becomes the input to the next. The same hidden-state stream is therefore processed by the shared Transformer within each iteration and carried across iterations as the recurrent state. Existing methods primarily address the limitations of recurrent depth in two ways: timestep encodings and loop-conditioning mechanisms provide shared layers with iteration identity (Xu and

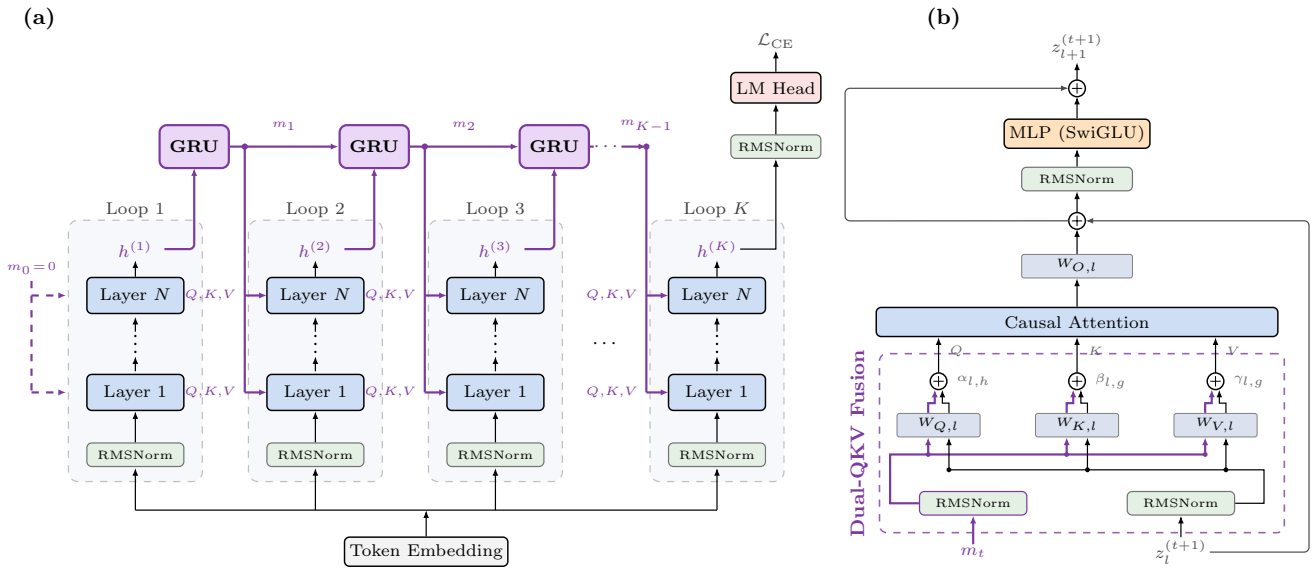


Figure 2: Overview of the RecLoop architecture. **(a)** A shared stack of N Transformer layers is unrolled for K iterations. Each iteration starts from the token embeddings; at each of the first $K-1$ loop boundaries, a shared GRU writes the terminal state $h^{(t)}$ into a token-aligned cross-loop memory m_t, which is exposed to every layer of the next iteration. **(b)** Dual-QKV Fusion within one RecLoop layer. The current working state $z_l^{(t+1)}$ provides the base Q, K, and V projections, while the separately normalized and projected cross-loop memory m_t is added to all three pathways through learned per-head gates; the current state also follows the standard attention and MLP residual paths. All Transformer and GRU parameters are shared across iterations, so increasing K does not increase the parameter count.

Sato 2025; Jeddi, Ciccone, and Taati 2026), while residual scaling, normalization, and stabilization methods control recurrent dynamics and optimization (Wang et al. 2026; Li et al. 2026b; Prairie et al. 2026; Li et al. 2026a). However, these methods do not change how historical information is carried: subsequent iterations still access preceding computation only through the current, continuously evolving hidden state. Although this state can propagate history in principle, it is simultaneously updated as the working representation of the current iteration. The two roles therefore remain coupled, which may make cross-iteration information more difficult to preserve and selectively access.

To this end, we introduce **RecLoop**, a recurrent-state modeling framework with a dedicated pathway for capturing and passing information across loop iterations, as shown in Figure 2. Concretely, we model information flow between consecutive iterations as a recurrent state sequence, where the state used by iteration $t+1$ is jointly determined by the terminal representation of iteration t and the state accumulated through earlier iterations. RecLoop parameterizes this transition with a residualized GRU shared across iterations and maintains a token-aligned recurrent state. To make the accumulated history directly accessible to subsequent iterations, we further introduce **Dual-QKV Fusion**, a history-injection mechanism that uses learned gates to fuse projections from the recurrent state and the current hidden representation into the attention queries, keys, and values. Both the GRU and the injection interface are shared across iterations, providing a dedicated pathway for updating and accessing cross-iteration history without increasing the parameter count as the number of loops grows.

We evaluate RecLoop through controlled pretraining comparisons spanning diverse looped language-modeling baselines, parameter scales from 71M to 1.06B, and recurrent depths from $K = 3$ to $K = 12$. In the iso-parameter comparison at the 310M scale, RecLoop achieves the best language-modeling results: relative to the strongest baseline, Parcae, it reduces held-out perplexity from 16.0691 to 15.9411 and LAMBADA perplexity from 33.18 to 30.35. RecLoop also achieves lower validation loss at every matched model scale, corresponding to approximately 11% fewer parameters at matched loss, and outperforms the corresponding vanilla looped model at every recurrent depth tested. These results demonstrate the effectiveness of explicit cross-iteration state modeling for looped language models. By changing how information is updated and accessed across iterations rather than how recurrent dynamics are stabilized, RecLoop introduces a design axis orthogonal to existing stabilization methods and offers new insights into the design of looped LLMs.

2 Related Work

Looped Transformers and Iteration Conditioning. Looped Transformers increase unrolled computational depth by repeatedly applying a shared set of layers. The Universal Transformer formulates repeated computation as iterative re-

finement with a shared transition function, while ALBERT demonstrates the broader parameter efficiency of cross-layer sharing (Dehghani et al. 2019; Lan et al. 2020). Subsequent work studies the ability of weight-shared Transformers to support programmable computation, iterative algorithms, and multi-step latent reasoning (Giannou et al. 2023; Yang et al. 2024; Saunshi et al. 2025). Recent language models apply the same principle to recurrent test-time computation and large-scale pretraining (Geiping et al. 2025; Zhu et al. 2025). Iteration encodings and loop-conditioned modulation further allow shared layers to adapt their behavior to the current step (Xu and Sato 2025; Jeddi, Ciccone, and Taati 2026). These mechanisms allow computation to vary across iterations, but do not separately parameterize how cross-iteration state is updated and accessed.

State Access and Recurrent Memory. Several methods expose information produced at earlier depths to later computation. Depth-Attention and AttnRes aggregate hidden states from multiple depths rather than using only the latest residual representation (Zeng et al. 2026; Kimi Team 2026). RMoE carries recurrent routing context across layers, but uses this state to condition expert selection rather than to represent full token content (Qiu et al. 2025). Recursive Transformers have also introduced routed buffers, gated memory banks, and additional memory tokens to preserve information across repeated computation (Yu et al. 2025; Frey et al. 2026; Sapunov 2026). These approaches differ in where earlier information is stored, how it is updated, and how subsequent computation retrieves it. RecLoop maintains a single fixed-size, token-aligned recurrent state with the same width as the model hidden states. A shared transition updates this state at loop boundaries. Attention layers in the next iteration read it while maintaining a separate working residual stream.

Stabilizing Looped Transformers. Training very deep Transformers is sensitive to normalization and residual-stream growth. Although Pre-Norm improves optimization stability, residual variance may accumulate with depth and later blocks can contribute progressively smaller updates (Xiong et al. 2020; Gromov et al. 2025; Sun et al. 2025). DeepNet rescales residual branches to stabilize very deep networks, while Mix-LN, HybridNorm, and SiameseNorm modify normalization placement or pathways to improve deep-layer optimization (Wang et al. 2024; Li, Yin, and Liu 2026; Zhuo et al. 2025; Li et al. 2026b). Looped Transformers face related optimization difficulties because shared layers are repeatedly reused along the unrolled depth. Residual Scaling, DeepLoop, and Parcae address these difficulties by scaling recurrent residual updates, adapting depth-scaling rules to repeated layers, or constraining recurrent dynamics (Wang et al. 2026; Li et al. 2026a; Prairie et al. 2026). These methods primarily control residual magnitude, normalization, or optimization along the unrolled depth. RecLoop addresses a different component of the computation by changing the state passed between iterations and how later attention accesses it.

3 Method

Looped Transformer Setup

Let x be an input token sequence and $e = \mathrm{Embed}(x) \in \mathbb{R}^{B\times S\times d}$ its token embeddings. A looped (weight-shared) Transformer consists of a shared stack $F_\theta = f_\theta^{(L)} \circ \cdots \circ f_\theta^{(1)}$ of L physical layers, applied for K iterations to give an unrolled depth of $L\times K$. We write $z_l^{(t)}$ for the residual-stream state at layer l of loop t and $h^{(t)} = z_L^{(t)}$ for its terminal state. A generic vanilla loop can be expressed as

$$\begin{aligned} z_0^{(1)} &= \mathcal{N}_{\mathrm{emb}}(e), \\ h^{(t)} &= F_\theta\left(z_0^{(t)}\right), \\ z_0^{(t+1)} &= \mathcal{B}\left(h^{(t)}\right), \end{aligned} \tag{1}$$

where $\mathcal{B}$ denotes the loop-boundary transformation of the chosen backbone. Thus the same boundary state serves both as the next loop's working representation and as the only carrier between iterations.

RecLoop instead separates the within-loop *working state* from a persistent *recurrent state*. Each loop reinitializes its working stream from the same token embeddings, while a token-aligned memory m_{t-1} summarizes preceding loops and conditions the current computation through attention. We initialize the pre-loop memory as $m_0 = \mathbf{0}$:

$$\begin{aligned} &m_0 = \mathbf{0}, \qquad z_0^{(t)} = \mathcal{N}_{\mathrm{emb}}(e), \\ &h^{(t)} = F_\theta\left(z_0^{(t)}; m_{t-1}\right). \end{aligned} \tag{2}$$

The zero memory contributes nothing to the first loop, which therefore reduces to standard self-attention. The working stream $z^{(t)}$ performs the layer-wise transformations within loop t, whereas m_{t-1} is dedicated to persistence along the loop axis. The complete RecLoop architecture exposes this memory to all three attention projections through Dual-QKV Fusion, as defined below.

Cross-Loop Recurrent State Modeling

At each of the first $K-1$ loop boundaries, the same shared GRU-style transition G_ϕ reads the current terminal state $h^{(t)}$ and the preceding memory m_{t-1} to write a new cross-loop memory m_t. This includes the first boundary, where the GRU reads the zero-initialized state m_0 and produces m_1 for the second loop:

$$m_t = G_\phi\left(h^{(t)}, m_{t-1}\right), \qquad 1 \le t \le K-1. \tag{3}$$

The resulting state m_{K-1} conditions the final loop through Dual-QKV Fusion; no additional memory update is needed after the final working state has been computed.

Concretely, we use a residualized GRU update. Let $b_t = \mathcal{N}_{\mathrm{new}}(h^{(t)})$, $c_{t-1} = \mathcal{N}_{\mathrm{mem}}(m_{t-1})$, and $u_t = [b_t; c_{t-1}]$. The transition is

$$\begin{aligned} &g_t = \sigma(W_g u_t + b_g), \qquad r_t = \sigma(W_r u_t + b_r), \\ &n_t = \tanh(W_n b_t + r_t \odot W_c c_{t-1} + b_n), \\ &w_t = b_t + W_o n_t, \qquad m_t = (1-g_t)\odot m_{t-1} + g_t \odot w_t. \end{aligned} \tag{4}$$

We initialize W_o to zero, so the transition initially recovers the direct write $w_t = b_t$ while retaining a learnable gated path for accumulating earlier states.

The recurrence is token-aligned: for each sequence position s, $m_t[:, s, :] = G_\phi(h^{(t)}[:, s, :], m_{t-1}[:, s, :])$, with all cross-token interaction left to causal attention. We use a full-width memory, $m_t \in \mathbb{R}^{B\times S\times d}$, so the attention interface is not confined to a fixed low-dimensional subspace. The shared GRU runs at each of the first $K-1$ loop boundaries, including the first boundary that maps $m_0 = \mathbf{0}$ to m_1, and is trained with full back-propagation through $m_{1:K-1}$ without stopping gradients through the previous memory.

Dual-QKV Fusion

The recurrent state must be exposed to the next loop through attention. Let $\widetilde{z}_l^{(t)}$ and $\widetilde{m}_{t-1,l}$ denote the current working state and preceding memory after the input normalization shared by physical layer l. We write the attention interface generally as

$$\left(Q_l^{(t)}, K_l^{(t)}, V_l^{(t)}\right) = \Phi_l\left(\widetilde{z}_l^{(t)}, \widetilde{m}_{t-1,l}\right). \tag{5}$$

When $t = 1$, the interface reads $m_0 = \mathbf{0}$, so the memory branches contribute zero and the computation reduces to ordinary self-attention. For all subsequent loops, RecLoop uses Dual-QKV Fusion as defined below.

Dual-QKV Fusion uses the working state as the base of all three projections and adds the recurrent-memory projections through learned per-head gates:

$$\begin{aligned} Q_l^{(t)} &= W_{Q,l}\widetilde{z}_l^{(t)} + \alpha_{l,h} W_{Q,l}\widetilde{m}_{t-1,l}, \\ K_l^{(t)} &= W_{K,l}\widetilde{z}_l^{(t)} + \beta_{l,g} W_{K,l}\widetilde{m}_{t-1,l}, \\ V_l^{(t)} &= W_{V,l}\widetilde{z}_l^{(t)} + \gamma_{l,g} W_{V,l}\widetilde{m}_{t-1,l}, \end{aligned} \tag{6}$$

where $\alpha_{l,h}$ gates the memory contribution per query head, while $\beta_{l,g}$ and $\gamma_{l,g}$ gate it per key/value head under grouped-query attention. This interface lets history modify both the attention addresses and the retrieved content while always preserving the projections of the representation refined within the current loop.

Language-modeling supervision is applied only after the final loop. Given the terminal state $h^{(K)}$, the logits and training objective are computed using the standard final normalization and language-modeling head:

$$y^{(K)} = W_{\mathrm{LM}}\mathcal{N}_{\mathrm{out}}(h^{(K)}), \qquad \mathcal{L} = \mathrm{CE}\left(y^{(K)}, y_{\mathrm{target}}\right). \tag{7}$$

Earlier loops receive no direct language-modeling supervision, but obtain gradients through the final loss and the full recurrent state chain. The shared residualized GRU adds $7d^2 + 5d$ parameters independently of K (about 12.7M at the main width $d = 1344$); the per-head gates used by the fusion interface contribute a negligible additional count.

4 Experiments

Experimental Setup

Data and Metrics. We pretrain all models from scratch on FineWeb-Edu (Penedo et al. 2024) with a sequence length of 2048 and apply the language-modeling loss only to the final loop. We report final-step training cross-entropy (CE), perplexity on a held-out FineWeb-Edu split (held-out PPL), and LAMBADA perplexity (LMB PPL). We additionally report 0-shot accuracy on seven downstream tasks: LAMBADA (Paperno et al. 2016), HellaSwag (Zellers et al. 2019), PIQA (Bisk et al. 2020), WinoGrande (Sakaguchi et al. 2020), ARC-easy and ARC-challenge (Clark et al. 2018), and OpenBookQA (Mihaylov et al. 2018). LAMBADA and WinoGrande use accuracy; the remaining tasks use length-normalized accuracy. Because several multiple-choice tasks are close to their random baselines at this scale, we use the three language-modeling metrics as primary evidence and the seven-task average as supporting evidence.

Baselines. We organize the comparison by the mechanism each baseline introduces. The first group is the *vanilla looped Transformer*, whose residual stream is passed directly between repeated applications of a shared stack. The second group controls residual growth or recurrent stability: Residual Scaling (Wang et al. 2026), DeepLoop (Li et al. 2026a), SiameseNorm (Li et al. 2026b), and Parcae (Prairie et al. 2026). The third group adds an explicit cross-layer or cross-loop mechanism: Depth-Attention (Zeng et al. 2026), Block AttnRes (Kimi Team 2026), timestep-modulated looped Transformers (TMLT) (Xu and Sato 2025), and MeSH (Yu et al. 2025). All methods are instantiated on the same looped language-modeling backbone rather than compared using numbers quoted from their original papers.

Training and Iso-Parameter Setup. Models are implemented based on the SmolLM (Allal et al. 2024) codebase and use L=12 physical Transformer layers unrolled for K=6 loops, for an effective depth of 72. The main scale uses d_{model}=1344, SwiGLU FFNs, 21 attention heads of dimension 64, 7 key/value heads, and a 49152-token SmolLM vocabulary with tied input and output embeddings. Table 1 compares models under an approximately 310M-parameter budget. When method-specific mechanisms introduce additional parameters, we adjust the FFN intermediate width as needed to align the total parameter count with this budget. Every run uses the same 10.0B-token budget, AdamW with $\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping at 1.0, a warmup-stable-decay schedule with 150 warmup steps, and a global batch of 1024 sequences. Main-scale runs use a peak learning rate of 1×10^{-3} and train for 4768 steps in bf16 with fp32 master weights and full back-propagation through the recurrent state chain. Held-out PPL is evaluated on 9700 unseen sequences, and downstream evaluation uses the lm-eval-harness (Gao et al. 2024) log-likelihood protocol.

Main Results

Table 1 compares RecLoop with the three baseline families under the shared protocol. Among completed baselines, Parcae gives the strongest language-modeling results, reaching CE 2.6654, held-out PPL 16.0691, and LAMBADA PPL 33.18. Residual Scaling also consistently improves over the vanilla loop, confirming that controlling recurrent residual dynamics is useful. However, the remaining gap to RecLoop indicates that stabilizing the residual stream and explicitly

	Language modeling			*Downstream (0-shot accuracy, %)*							
Method	CE↓	PPL↓	LMB PPL↓	LAMBADA	HellaSwag	PIQA	WinoGrande	ARC-e	ARC-c	OpenBookQA	Avg↑
Reference											
Vanilla looped	2.7073	16.85	42.88	32.25	40.19	67.36	51.54	54.67	29.10	32.40	43.93
Residual control and recurrent stability											
Residual Scaling	2.6840	16.43	38.17	33.61	41.21	66.65	51.54	54.67	**30.12**	**35.20**	44.71
DeepLoop	7.5138	1992.8	2.8×10^6	0.00	26.02	48.69	49.57	27.36	24.32	27.80	29.11
SiameseNorm	2.6792	16.348	35.04	34.58	41.13	66.43	52.49	54.46	29.01	34.00	44.59
Parcae	2.6654	16.0691	33.18	35.67	**42.26**	66.54	51.54	54.08	28.41	34.00	44.64
Cross-layer and cross-loop mechanisms											
Depth-Attention	2.6871	16.496	39.43	32.74	40.72	66.70	50.51	54.42	29.10	34.80	44.14
Block AttnRes	2.7347	17.3744	45.89	31.92	38.39	66.76	50.28	**55.13**	27.82	**35.20**	43.64
TMLT	2.6962	16.6448	40.45	32.93	40.59	67.46	51.22	53.79	29.27	33.80	44.15
MeSH	2.6778	16.354	35.62	34.76	41.25	66.70	51.54	53.70	29.10	33.60	44.38
Ours											
RecLoop	**2.6567**	**15.9411**	**30.35**	**36.41**	41.92	**67.85**	**53.28**	**55.13**	28.67	34.40	**45.38**

Table 1: Iso-parameter comparison of looped language models pretrained from scratch on FineWeb-Edu for 10B tokens. All models use a 12-layer shared stack unrolled for K=6 loops. We report 0-shot accuracy on seven downstream tasks and their average. Best and second-best results are shown in **bold** and underlined, respectively. DeepLoop does not converge under the shared recipe and is shown for completeness.

carrying history are complementary rather than interchangeable.

RecLoop achieves the best language-modeling quality, with CE 2.6567, held-out PPL 15.9411, and LAMBADA PPL 30.35. Relative to the strongest completed baseline, Parcae, these correspond to relative reductions of 0.33%, 0.80%, and 8.53%, respectively. Its seven-task average accuracy is 45.38%; because the changes vary across individual tasks at this model scale, we interpret the aggregate downstream result as supporting evidence rather than claiming uniform task-wise dominance.

DeepLoop does not converge under the shared peak learning rate: its final CE is close to the FineWeb-Edu unigram entropy and its downstream scores remain near the random floor. We retain the run for completeness and interpret it only as a failure of this configuration under the shared recipe, not as a general assessment of DeepLoop.

Parameter Scaling

Figure 1 reports the parameter-scaling sweep under a fixed 10B-token training budget. As model size increases from 71M to 1.06B, RecLoop maintains a lower validation loss than the vanilla looped model at every matched scale; the joint fit corresponds to approximately 11% fewer parameters at matched loss. The language-modeling advantage of the RecLoop architecture is therefore not confined to a single parameter scale.

Loop-Depth Scaling

To test whether the advantage of the RecLoop architecture depends on a particular recurrent depth, we fix a 12-layer physical stack and a 10B-token training budget at the 310M-parameter scale, vary the loop count from $K = 3$ to $K = 12$, and train a matched Vanilla Looped LM and RecLoop at every K. Figure 3 shows that RecLoop achieves lower terminal training cross-entropy, FineWeb-Edu held-out perplexity, and LAMBADA perplexity at all four recurrent depths. Its absolute FineWeb-Edu PPL advantage is $0.556, 0.550, 0.613$, and 0.585 at $K = 3, 6, 9, 12$, respectively, indicating that the benefit of RecLoop cross-loop state modeling remains stable across unrolled depths rather than appearing only at a particular value of K.

Increasing the loop count generally improves held-out language modeling, although the improvements on LAMBADA saturate. The FineWeb-Edu PPL of RecLoop drops substantially from $K = 3$ to $K = 6$, followed by progressively smaller gains at $K = 9$ and $K = 12$; its LAMBADA PPL is lowest at $K = 9$ and is effectively unchanged at $K = 12$. The vanilla baseline exhibits the same saturation trend, and its LAMBADA perplexity worsens at $K = 12$. These results support a robust but bounded conclusion: RecLoop does not remove the saturation of recurrent depth, but consistently shifts the language-modeling curves below the vanilla baseline throughout the evaluated range, allowing both shallower and deeper looped configurations to benefit from explicit cross-loop state.

Does RecLoop Accumulate and Use History?

We perform inference-time causal knockouts on the 310M RecLoop checkpoint without retraining. The recurrent state is token-aligned, so at boundary s we independently perturb either the current content $h^{(s)}$ or the incoming recurrent state m_{s-1} by permuting token positions while preserving the multiset of state vectors and their norms. This intervention breaks the correspondence between state content and token position without changing the multiset of state vectors, allowing us to test whether the final prediction depends

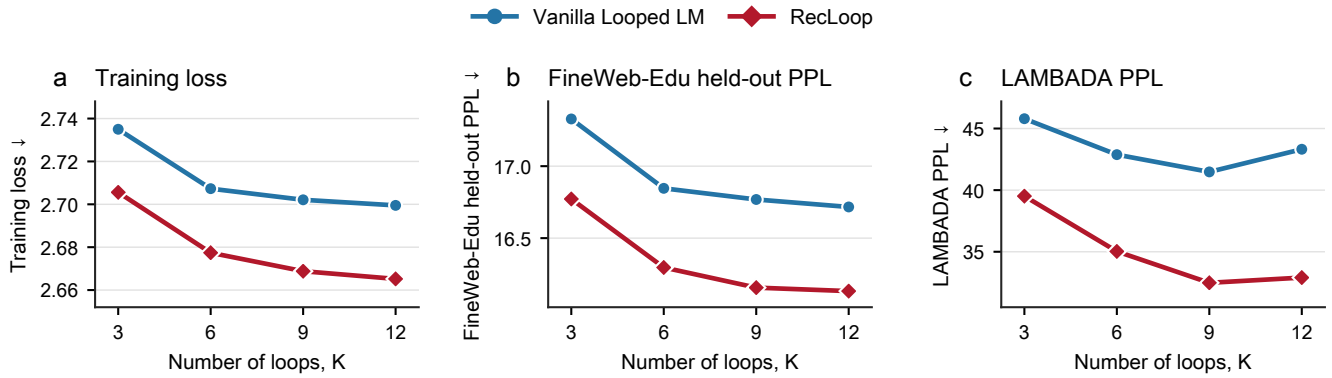


Figure 3: Loop-depth scaling at the 310M-parameter scale. All models use the same 12-layer physical stack and are trained on 10B tokens. Varying the loop count K increases the unrolled depth from 36 to 144. RecLoop achieves lower training loss, FineWeb-Edu held-out PPL, and LAMBADA PPL at every K. Both models exhibit diminishing returns as K grows, while RecLoop retains a stable language-modeling advantage throughout the sweep.

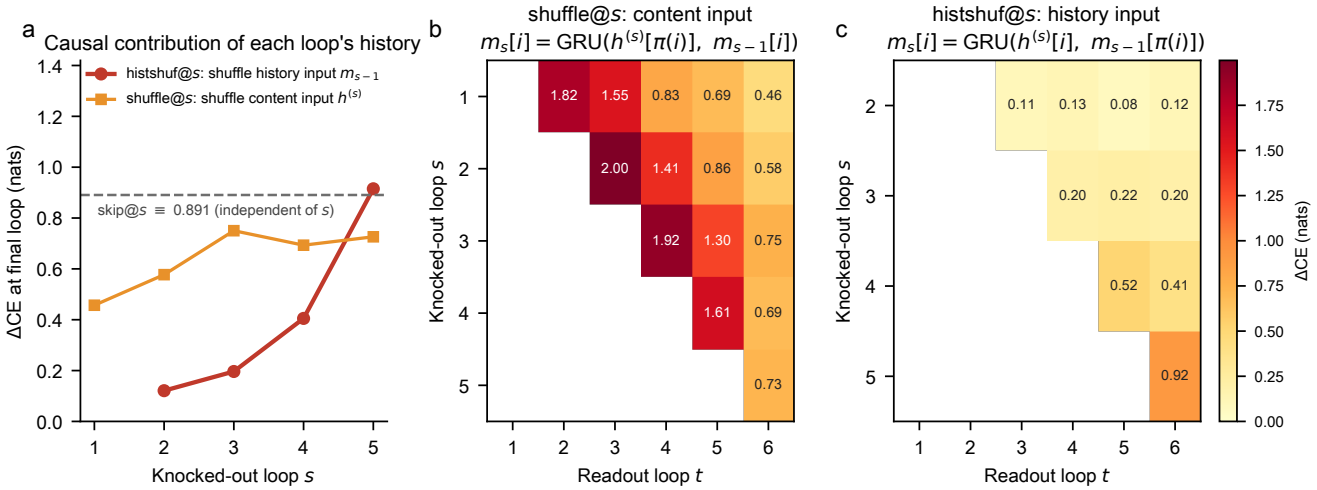


Figure 4: History knockouts on the 310M RecLoop model. Perturbing the current or recurrent state increases final-loop CE, showing that final predictions depend on token-aligned content carried by the recurrent state.

on the token-aligned content carried by the recurrent-state pathway.

Figure 4 shows that shuffling the recurrent state at boundaries $s = 2, 3, 4, 5$ increases final-loop CE by 0.12, 0.20, 0.41, and 0.92 nats, respectively. The positive effect at every tested boundary shows that the final prediction causally depends on token-aligned content carried by the recurrent state. The effect decreases approximately with distance from the final loop, suggesting that the model relies more strongly on recent recurrent states.

Component Ablations

All component ablations use the same 12-layer, six-loop backbone and training budget unless stated otherwise, with the full RecLoop model using Dual-QKV Fusion as the reference architecture. We remove or replace one component at a time and use CE, held-out PPL, and LAMBADA PPL as the primary criteria.

Where Should Memory Enter Attention? This ablation starts from Dual-QKV Fusion in the full RecLoop model and disables the gated memory contribution to Q, K, or V while retaining the corresponding current-state projection in every arm. Table 2 therefore marks which memory pathways remain enabled. Among these reduced fusion variants, the full Dual-QKV Fusion mechanism performs best on all four reported metrics: compared with Dual-QK, which removes the memory value pathway, it reduces held-out PPL from 16.061 to 15.9411 and LAMBADA PPL from 33.07 to 30.35, while increasing the seven-task average from 44.69 to 45.38. We consequently use Dual-QKV Fusion in the final RecLoop architecture.

Variant	Mem-Q	Mem-K	Mem-V	CE↓	PPL↓	LMB PPL↓	Avg↑
Dual-Q	✓	×	×	2.6774	16.296	35.03	44.61
Dual-K	×	✓	×	2.7144	16.9167	42.64	43.39
Dual-QK	✓	✓	×	2.6645	16.061	33.07	44.69
Dual-V	×	×	✓	2.7150	16.940	40.09	44.25
Dual-QKV Fusion	✓	✓	✓	**2.6567**	**15.9411**	**30.35**	**45.38**

Table 2: Attention-pathway ablation of Dual-QKV Fusion at the 310M scale. A checkmark retains the gated recurrent-memory contribution to the corresponding current-state Q, K, or V projection; a cross disables only that memory contribution. All variants share the GRU carrier, parameter count, computation, data, and training recipe.

What Should Carry Information Across Loops? Table 3 fixes Dual-QKV Fusion in every RecLoop variant, varying only the carrier that constructs the cross-loop state to separate carrier capacity from recurrent history. Replacing the parameter-free carrier with a learned boundary MLP improves CE from 2.6865 to 2.6648 and PPL from 16.46 to 16.06, showing that a learned carrier is useful even without accumulated history. The full RecLoop variant with the recurrent GRU further improves CE, held-out PPL, and LAMBADA PPL to 2.6567, 15.9411, and 30.35, respectively, compared with 2.6648, 16.06, and 36.26 for the boundary MLP and 2.6736, 16.2063, and 36.18 for the parameter-matched memory-zeroed GRU. Thus, learning the carrier is broadly useful, while accumulated recurrent history provides additional gains across all three language-modeling metrics after matching parameters, computation, and nonlinearity.

Carrier	ΔParams	Prev.	Hist.	CE↓	PPL↓	LMB PPL↓
(1) Parameter-free	0	✓	×	2.6865	16.46	37.13
(2) Boundary MLP	$7d^2+d$	✓	×	2.6648	16.06	36.26
(3) 0-memory GRU	$7d^2+5d$	✓	×	2.6736	16.2063	36.18
(4) Full GRU	$7d^2+5d$	✓	✓	**2.6567**	**15.9411**	**30.35**

Table 3: Cross-loop carrier ablation for RecLoop at the 310M scale. All arms share the backbone, Dual-QKV Fusion, training recipe, and token budget. The boundary MLP tests learned carrier capacity without recurrence; the memory-zeroed and full GRU arms are matched in parameters and computation, isolating access to accumulated history.

5 Conclusion

We introduced RecLoop, a recurrent state modeling framework that separates persistent cross-loop memory from the working representation of each iteration. RecLoop accumulates token-aligned history through a shared residualized GRU and exposes it to subsequent iterations through gated Dual-QKV attention. In controlled pretraining on FineWeb-Edu, RecLoop achieved the best language-modeling results among the evaluated methods at the 310M-parameter scale and outperformed matched vanilla looped models across all tested model scales from 71M to 1.06B and loop counts from $K = 3$ to $K = 12$; the fitted scaling laws indicate approximately 11% fewer parameters at matched validation loss. Causal history perturbations show that later iterations depend on accumulated history, while component ablations favor injecting memory into Q, K, and V. These results establish explicit cross-loop state modeling as an effective and complementary way to organize recurrent computation.

A Implementation Details

Table 4 summarizes the shared configuration for the main experiments at the 310M parameter scale. All models are trained from scratch.

Setting	Value
Data	
Training corpus	FineWeb-Edu (Penedo et al. 2024)
Tokenizer / vocabulary	SmolLM / 49,152
Sequence length	2048
Training budget	10B tokens
Held-out evaluation	9700 sequences
Model architecture	
Physical layers / loops	$12/6$ (effective depth 72)
Hidden / FFN width	$1344/3584$
Attention heads / KV heads	$21/7$
Head dimension	64
Memory width	1344
Optimization	
Optimizer	AdamW
Adam coefficients	$(0.9, 0.95)$
Weight decay	0.1
Peak learning rate	1×10^{-3}
Warmup steps	150
Global batch size	1024
Gradient clipping	1.0
Precision	bf16 computation, fp32 parameters

Table 4: Implementation settings for the main experiments.

Experiments use MetaX C550 64GB GPUs, PyTorch 2.8, and Transformers 4.52.4.

B Training and Evaluation Details

Table 5 lists the setup shared by the supplementary controls at the 310M scale. All runs train from scratch on 10B FineWeb-Edu tokens and use checkpoint 4768. Only the final loop is supervised, with full back-propagation. Embeddings and the LM head are tied; we use no attention dropout, intermediate-loop loss, or loop-specific conditioning.

Setting	Value
Corpus / tokenizer	FineWeb-Edu / SmolLM
Vocabulary / sequence length	49,152/2048
Training budget / steps	10B tokens / 4768
Physical layers / loops	12/6
Hidden / FFN width	1344/3584
Query / key-value heads	21/7
Head / memory width	64/1344
Optimizer	AdamW, $\beta = (0.9, 0.95)$
Peak LR / warmup	1×10^{-3}/150 steps
Weight decay / grad. clip	0.1/1.0
Global batch size	1024 sequences
Precision	bf16 compute, fp32 master weights
QK-Norm / auxiliary loss	disabled / disabled

Table 5: Shared setup for the reported 310M-scale controls.

ID	Carrier	Train CE	Held CE	PPL	LMB PPL
N1	Direct	2.6914	2.8071	16.5618	39.1290
N2	Stateless GRU	2.6867	2.8023	16.4825	38.3755
N3	Recurrent GRU	**2.6822**	**2.7968**	**16.3921**	**37.9344**

Table 6: Carry carrier controls with Dual-QKV; bold marks the best result.

Held-out CE is token-weighted over 9700 complete 2048-token blocks from an unseen FineWeb-Edu shard; PPL is $\exp(\mathrm{CE})$, and LAMBADA PPL is zero-shot. As multiple-choice tasks remain near random baselines, we focus on these language-modeling metrics.

C Additional Carry-Stream Controls

The *Carry* stream starts from embeddings and then passes each raw terminal state forward. We compare Direct, with no independent state update; Stateless GRU, with previous memory zeroed; and Recurrent GRU, which retains it. All runs share the Dual-QKV interface, 12×6 backbone, data, budget, and protocol.

Every metric improves from N1 to N2 to N3. Relative to N2, N3 lowers held-out CE, PPL, and LAMBADA PPL by 0.0055, 0.0904, and 0.4411. With one seed per arm, this is supplementary evidence of a marginal recurrent-edge benefit.

D Extended Memory-Injection Ablation

Table 7 fixes the Recurrent GRU and Restart working stream, varying only the enabled memory pathways while retaining all current-state projections.

Variant	Mem-Q	Mem-K	Mem-V	Train CE↓	PPL↓	LMB PPL↓
Dual-Q	✓	×	×	2.6774	16.2960	35.03
Dual-K	×	✓	×	2.7144	16.9167	42.64
Dual-QK	✓	✓	×	2.6645	16.0610	33.07
Dual-V	×	×	✓	2.7150	16.9400	40.09
Dual-KV	×	✓	✓	2.6843	16.4200	35.9286
Dual-QKV	✓	✓	✓	**2.6567**	**15.9411**	**30.35**

Table 7: Memory-injection pathway ablation at the 310M scale.

Q is the strongest single pathway. Dual-KV beats K-only and V-only but not Q-only, showing K/V complementarity. QK further improves over Q-only; adding V then lowers held-out PPL from 16.0610 to 15.9411 and LAMBADA PPL from 33.07 to 30.35. Full Dual-QKV is therefore best overall.

E Limitations

Structural controls use one seed per arm, so small differences are marginal evidence. Experiments are limited to 10B-token FineWeb-Edu base pretraining.

References

Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2024. SmolLM - blazingly fast and remarkably powerful.

Bisk, Y.; Zellers, R.; Le Bras, R.; Gao, J.; and Choi, Y. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; and Tafjord, O. 2018. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Dehghani, M.; Gouws, S.; Vinyals, O.; Uszkoreit, J.; and Kaiser, L. 2019. Universal Transformers. In *International Conference on Learning Representations*.

Frey, M.; Shomali, B.; Bashir, A. H.; Berghaus, D.; Koehler, J.; and Ali, M. 2026. Adaptive Loops and Memory in Transformers: Think Harder or Know More? *arXiv preprint arXiv:2603.08391*. Latent and Implicit Thinking Workshop at ICLR 2026.

Gao, L.; Tow, J.; Abbasi, B.; Biderman, S.; Black, S.; DiPofi, A.; Foster, C.; Golding, L.; Hsu, J.; Le Noac'h, A.; Li, H.; McDonell, K.; Muennighoff, N.; Ociepa, C.; Phang, J.; Reynolds, L.; Schoelkopf, H.; Skowron, A.; Sutawika, L.; Tang, E.; Thite, A.; Wang, B.; Wang, K.; and Zou, A. 2024. The Language Model Evaluation Harness.

Geiping, J.; McLeish, S.; Jain, N.; Kirchenbauer, J.; Singh, S.; Bartoldson, B.; Kailkhura, B.; Bhatele, A.; and Goldstein, T. 2025. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *Advances in Neural Information Processing Systems*, 38: 41340–41391.

Giannou, A.; Rajput, S.; Sohn, J.-y.; Lee, K.; Lee, J. D.; and Papailiopoulos, D. 2023. Looped Transformers as Programmable Computers. In *International Conference on Machine Learning*.

Gromov, A.; Tirumala, K.; Shapourian, H.; Glorioso, P.; and Roberts, D. 2025. The Unreasonable Ineffectiveness of the Deeper Layers. In *The Thirteenth International Conference on Learning Representations*.

Hoffmann, J.; Borgeaud, S.; Mensch, A.; Buchatskaya, E.; Cai, T.; Rutherford, E.; de Las Casas, D.; Hendricks, L. A.; Welbl, J.; Clark, A.; Hennigan, T.; Noland, E.; Millican, K.; van den Driessche, G.; Damoc, B.; Guy, A.; Osindero, S.; Simonyan, K.; Elsen, E.; Rae, J. W.; Vinyals, O.; and Sifre, L. 2022. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*.

Jeddi, A.; Ciccone, M.; and Taati, B. 2026. LoopFormer: Elastic-Depth Looped Transformers for Latent Reasoning via Shortcut Modulation. *arXiv preprint arXiv:2602.11451*.

Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T. B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; and Amodei, D. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*.

Kimi Team. 2026. Attention Residuals. *arXiv preprint arXiv:2603.15031*.

Lan, Z.; Chen, M.; Goodman, S.; Gimpel, K.; Sharma, P.; and Soricut, R. 2020. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations*.

Li, P.; Yin, L.; and Liu, S. 2026. Mix-LN: Unleashing the Power of Deeper Layers by Combining Pre-LN and Post-LN. In *The Thirteenth International Conference on Learning Representations*.

Li, S.; Zhang, Y.; Guo, J.; Gu, Q.; and Wang, M. 2026a. DeepLoop: Depth Scaling for Looped Transformers. *arXiv preprint arXiv:2607.13491*.

Li, T.; Han, D.; Cao, Z.; Huang, H.; Zhou, M.; Chen, M.; Zhao, E.; Jiang, X.; Jiang, G.; and Huang, G. 2026b. SiameseNorm: Breaking the Barrier to Reconciling Pre/Post-Norm. *arXiv preprint arXiv:2602.08064*.

Mihaylov, T.; Clark, P.; Khot, T.; and Sabharwal, A. 2018. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.

Paperno, D.; Kruszewski, G.; Lazaridou, A.; Pham, Q. N.; Bernardi, R.; Pezzelle, S.; Baroni, M.; Boleda, G.; and Fernández, R. 2016. The LAMBADA Dataset: Word Prediction Requiring a Broad Discourse Context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*.

Penedo, G.; Kydlíček, H.; Lozhkov, A.; Mitchell, M.; Raffel, C. A.; von Werra, L.; Wolf, T.; et al. 2024. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *Advances in Neural Information Processing Systems*, 37.

Prairie, H.; Novack, Z.; Berg-Kirkpatrick, T.; and Fu, D. Y. 2026. Parcae: Scaling Laws for Stable Looped Language Models. *arXiv preprint arXiv:2604.12946*.

Qiu, Z.; Huang, Z.; Cheng, S.; Zhou, Y.; Wang, Z.; Titov, I.; and Fu, J. 2025. Layerwise Recurrent Router for Mixture-of-Experts. In *International Conference on Learning Representations*.

Sakaguchi, K.; Le Bras, R.; Bhagavatula, C.; and Choi, Y. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Sapunov, G. 2026. Universal Transformers Need Memory: Depth-State Trade-offs in Adaptive Recursive Reasoning. *arXiv preprint arXiv:2604.21999*.

Saunshi, N.; Dikkala, N.; Li, Z.; Kumar, S.; and Reddi, S. J. 2025. Reasoning with Latent Thoughts: On the Power of Looped Transformers. *arXiv preprint arXiv:2502.17416*.

Sun, W.; Song, X.; Li, P.; Yin, L.; Zheng, Y.; and Liu, S. 2025. The curse of depth in large language models. *Advances in Neural Information Processing Systems*, 38: 163104–163136.

Wang, H.; Ma, S.; Dong, L.; Huang, S.; Zhang, D.; and Wei, F. 2024. DeepNet: Scaling Transformers to 1,000 Layers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Wang, S.; Li, B.; Zhang, G.; Huang, W.; and Li, J. 2026. On the Residual Scaling of Looped Transformers: Stability and Transferability. In *ICLR 2026 Workshop on Latent and Implicit Thinking*.

Xiong, R.; Yang, Y.; He, D.; Zheng, K.; Zheng, S.; Xing, C.; Zhang, H.; Lan, Y.; Wang, L.; and Liu, T.-Y. 2020. On Layer Normalization in the Transformer Architecture. In *International Conference on Machine Learning*, 10524–10533.

Xu, K.; and Sato, I. 2025. On Expressive Power of Looped Transformers: Theoretical Analysis and Enhancement via Timestep Encoding. In *International Conference on Machine Learning*, 69613–69646. PMLR.

Yang, L.; Lee, K.; Nowak, R. D.; and Papailiopoulos, D. 2024. Looped Transformers are Better at Learning Learning Algorithms. In *International Conference on Learning Representations*.

Yu, C.; Shu, X.; Wang, Y.; Zhang, Y.; Wu, H.; Li, J.; Long, R.; Chen, Z.; Xu, Y.; Su, W.; et al. 2025. MeSH: Memory-as-State-Highways for Recursive Transformers. *arXiv preprint arXiv:2510.07739*.

Zellers, R.; Holtzman, A.; Bisk, Y.; Farhadi, A.; and Choi, Y. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Zeng, B.; Hao, Y.; Wang, Z.; Song, S.; Li, H.; Song, F.; Liu, Y.; He, Z.; Wang, X.; and Lin, Z. 2026. Depth-Attention: Cross-Layer Value Mixing for Language Models. *arXiv preprint arXiv:2606.05014*.

Zhu, R.-J.; Wang, Z.; Hua, K.; Zhang, T.; et al. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

Zhuo, Z.; Zeng, Y.; Wang, Y.; Zhang, S.; Li, X.; Yang, J.; Ma, J.; et al. 2025. Hybridnorm: Towards stable and efficient transformer training via hybrid normalization. *Advances in Neural Information Processing Systems*, 38: 25106–25147.